

Intuition of the VAE

The most important part to read is to figure out how training works is 4.

Implementation: Reparameterization after that its just standard gradient based optimization that you use with normal nns

1. Encoder

- Maps each individual input to its own distribution. (so every image would have its very own distribution that corresponds to it)
 - For every image, outputs a mean (μ) and a standard deviation (σ) (this is specific to that particular image)
 - The thing that it outputs is called the latent space (denoted by N), it is essentially two arrays both of length N , the first one is for all the means, and the second one is for all the sigmas. So you have N standard deviations to describe the data. (This is how complexity is captured)

2. Decoder

- Draws a random sample from the normal distribution that the encoder just generated ($z \sim (\mu, \sigma)$) and tries to reconstruct the original image from it

3. Loss

- Balancing act - this is where the variational comes from
- How close the reconstructed image is to the real image (if this was the only loss being used, it would just be a standard autoencoder (not variational, and it would just spit out the same image every time, which is not good for data augmentation)
- Two part loss:
 - reconstruction loss (standard loss such as MSE, BCE) punishment if the decoded image does not look like the original input.
 - Kullback-Leibler divergence - Math penalty that forces the μ and σ to sit as close to a normal distribution as possible (mean of 0 and a standard deviation of 1)

Total loss is: $\mathbb{L} = \text{Reconstruction Loss} + D_{KL}N(\mu, \sigma^2) \mid N(0, 1)$

- KL Term acts as a regularizer, ensures latent space is tightly packed and smooth, so when you run the sample points (as in you use the decoder to produce images), you don't get wonky numbers because your normal distribution isn't smooth)

4. Implementation: Reparameterization

- Cannot backpropagate gradients through a random sampling process, so you have to use something called reparameterization
- You apply the randomness afterwards by isolating the randomness, so you can still have gradients that backpropagation can work with
Typically, the VAE outputs μ (mean) of the distribution and the log variance (not sure why its not just the variance, apparently its because the variance must be positive, and this lets the network output any real number, so we are essentially preventing the network from being clipped).

Then, you would sample from this distribution (in a real application), reparameterization separates the sampling from the values so backpropagation can still work.

- Not too sure about how the math works, but essentially, you let ϵ be a sample from the standard distribution ($\epsilon \sim \mathcal{N}(0, 1)$ (I can't right the squiggly N for some reason))

This is where our noise comes from. To get from the normal distribution to the distribution defined by the variational autoencoder, you multiply by the standard deviation and add the mean. And that gets you to where it would be as if you had sampled it directly from the original distribution.

- Gradients propagate through out standard distribution (which we know the gradient of I'm assuming?), the sampling is transformed into a differentiable transformation so we can find the gradient of it. ϵ is random noise

- This is the very important equation. Unlucky for me, I can't read: $z = \mu + \sigma * \epsilon$ where $\epsilon \sim \mathcal{N}(0, 1)$

Inference

After the decoder has been trained, we provide it a vector of variables (they are called the **latent variables** in this case), and it generates an image that the decoder believes will fit the vector that we provided.

- The encoder gets completely thrown out in the inference stage (it is only used during the training phase)

